SISTEMA DE GESTÃO DE SERVIÇOS

Licenciatura de Engenharia Informática
Engenharia de Software
Docente: João Rebelo

Celso Batista, 90112
Porto, 2026

Conteúdo

Problema 1
Objetivos 2
Estado da Arte 3
Soluções Existentes e Análise de Mercado 3
Navia: A Referência no Setor das Águas 3
Soluções ERP Globais: SAP Business One e Microsoft Dynamics 4
Conclusão da Análise 4
Arquitetura de Sistema: Monólito Modular vs. Microserviços 5
Fundamentação Tecnológica 6
Análise Operacional: Processos Manuais vs. Automatização 7
Metodologia 8
Desenvolvimento 8
Engenharia de Requisitos (SRS) 9
1. Público-Alvo e Sugestões de Leitura 9
2. Âmbito do Projeto 9
3. Glossário 10
4. Referências 12
5. Organização deste Documento 14
6. Classes de Utilizadores 15
7. Funcionalidades do Produto 17
8. Ambiente de Operação 18
9. Restrições de Desenho e Implementação 18
10. Documentação para os Utilizadores 19
11. Pressupostos e Dependências 19
12. Interfaces de Utilizador 20
13. Interfaces de Hardware 21
14. Interfaces de Software 21
15. Interfaces de Comunicação 22
16. Requisitos Funcionais do Sistema 22

1. Visão de Gestão de Clientes 23
2. Visão de Gestão de Serviços e Agenda 24
3. Visão Operacional (Máquina de Estados) 25
4. Visão de Gestão de Frota (Quilometragem) 26
5. Visão de Gestão de Frota (Manutenções e Inspeções) 27
6. Visão Financeira e Faturação 28
7. Visão de Segurança e Acessos 29
8. Visão de Auditoria 30
17. Requisitos Não-Funcionais 31
18. Requisitos Não-Funcionais das Regras de Negócio 33
19. Outros Requisitos 34
20. Lista de Itens a Elaborar na Fase de Desenho 34
21. Continuidade do Processo de Desenvolvimento 35
22. Cronograma Proposto 36
23. Plano de Projeto do Sistema de Gestão de Serviços 36
Arquitetura de Software 38
Diagrama Use Case 38
Diagrama de Classes 39
Diagrama Entidade Relação (ER) 40
Construção e testes de software 41
Classe GeradorBenchmark 42
Controlador ClienteController 43
View Resultados do Teste de Carga 44
Conclusão 45

Tabelas

Tabela 1 - Análise Processos Manuais vs. Automatização 7
Tabela 2 – Glossário 11
Tabela 3 - Descrição dos Atores e Perfis de Acesso do Sistema 15
Tabela 4 - Perfis de Acesso e Níveis de Privilégio do Sistema. 15
Tabela 5 - Divisão modular proposta para o Sistema de Informação 16
Tabela 6 - RF1. Gestão de Clientes 23
Tabela 7 - RF2. Gestão de Serviços e Agenda 24
Tabela 8 - RF3. Máquina de Estados 25
Tabela 9 - RF4. Gestão de Frota (Quilometragem) 26
Tabela 10 - RF5. Gestão de Frota (Manutenções e Inspeções) 27
Tabela 11 - RF6. Visão Financeira e Faturação 28
Tabela 12 - RF7. Segurança e Acessos 29
Tabela 13 - RF8. Auditoria de Ações e Sistema 30
Tabela 14 - Requisitos Não-Funcionais 33
Tabela 15 - Cronograma Proposto 36

Figuras

Figura 1 - Diagrama de Arquitetura do Sistema 17
Figura 2 - Diagrama de Use Case 38
Figura 3 - Diagrama de Classes 39
Figura 4 - Diagrama de Entidade Relação 40
Figura 5 - Construção *Class* para Testes 42
Figura 6 - Construção *Controller* para Testes 43
Figura 7 - Teste de inserção de dados na base de dados 44

Problema

A Palco d'Água, fundada em 2006 no setor do saneamento mecanizado, enfrenta um ponto de inflexão tecnológica crítico devido à elevada dependência de processos manuais. O problema reside na gestão assente em métodos analógicos, como folhas de serviço em papel e agendas telefónicas, o que resulta numa baixa maturidade sistémica, silos de informação e redundância administrativa.

Esta ausência de uma plataforma centralizada impede o acompanhamento do ciclo de vida dos serviços em tempo real, dificultando a gestão de intervenções realizadas, faturadas ou pendentes. A desorganização é agravada pela "dupla escrita" — o registo manual no terreno seguido de transcrição para o sistema — o que gera carga administrativa excessiva e riscos de erro em dados sensíveis como NIF ou moradas.

No plano logístico, a falta de monitorização automatizada da frota eleva os custos operacionais, pois a ausência de alertas preventivos expõe a empresa a reparações urgentes. Este défice tecnológico culmina na estagnação da faturação, limitando a captação de clientes via canais digitais e impedindo a otimização da capacidade produtiva interna através da automação de processos.

Objetivos

Objetivo Geral: Conceber e implementar uma plataforma digital composta por um website institucional e um sistema de gestão operacional (CRM) que modernize o fluxo de trabalho da Palco d'Água, garantindo a centralização da informação e a melhoria da eficiência administrativa.

Objetivos Específicos (SMART):

1. **Implementar um sistema de autenticação** com três perfis de acesso (Admin, Financeiro e Funcionário), garantindo a segurança e o controlo de permissões.
2. **Digitalizar a gestão de serviços** através de uma agenda dinâmica com uma máquina de estados automatizada (Agendado, A decorrer, Para faturação, Faturado, A Aguardar Pagamento, Pago, Cancelado).
3. **Implementar um sistema híbrido de preenchimento automático de moradas**, priorizando a consulta a um repositório local e recorrendo à GeoAPI.pt apenas em caso de ausência de dados, persistindo o resultado para otimização de recursos.
4. **Criar um módulo de gestão de frota** que registe quilometragens e emita alertas automáticos para revisões.
5. **Garantir a persistência e integridade dos dados** utilizando uma base de dados MariaDB alojada no servidor atual da empresa.

Âmbito do Projeto:

O projeto inclui a criação do ERP em subdomínio privado. O ERP gerirá o ciclo de vida do serviço: o funcionário inicia e termina a intervenção no portal; o sistema questiona a necessidade de faturação e o estado do pagamento, transitando o estado do serviço automaticamente. Inclui também o arquivo digital de faturas (PDFs externos) e a gestão de estados financeiros (A Aguardar Pagamento vs. Pago).

Limites do Âmbito:

<u>Não inclui:</u>

- Integração direta via API com o software de faturação da empresa (emissão externa);
- Gateways de pagamento automático (Multibanco/MBWay).

Estado da Arte

O desenvolvimento de sistemas de informação para a gestão operacional (CRM/ERP) tem evoluído de estruturas monolíticas rígidas para arquiteturas modulares e reativas. Este capítulo analisa o panorama atual, fundamenta as opções arquiteturais e tecnológicas, e compara as soluções de mercado com as necessidades específicas de uma microempresa.

Soluções Existentes e Análise de Mercado

A transformação digital impõe-se atualmente como um vetor crítico para a competitividade empresarial. Segundo Ulas (2019), a digitalização nas Pequenas e Médias Empresas (PMEs) permite não só a otimização de processos, mas também a criação de novos modelos de valor. Contudo, a literatura identifica barreiras significativas à adoção de tecnologias nestas organizações, nomeadamente a escassez de recursos financeiros e a falta de competências digitais internas (Rupeika-Apoga & Petrovska, 2022).

Neste contexto, a análise de mercado focou-se em três soluções de gestão proeminentes — Navia, SAP Business One e Microsoft Dynamics 365 — avaliando a sua adequação à realidade específica da Palco d'Água.

Navia: A Referência no Setor das Águas

O Navia posiciona-se como uma plataforma especializada para o setor dos serviços públicos essenciais (água, energia e resíduos), sendo amplamente utilizada por grandes gestoras de infraestruturas em Portugal.

- **Aspetos Positivos:** Destaca-se pela sua capacidade de integração vertical, unificando dados provenientes de sistemas SCADA (telemetria), GIS (sistemas de informação geográfica) e equipas de terreno numa única interface de comando. É uma solução robusta para a monitorização de ativos em larga escala.
- **Limitações para a Palco d'Água:** A sua arquitetura foi desenhada para organizações com departamentos segmentados (Manutenção, Planeamento, Qualidade). Para uma microempresa onde a gestão é centralizada numa única pessoa, o fluxo de trabalho do Navia revela-se excessivamente burocrático e sobredimensionado, exigindo

configurações complexas que não trazem retorno imediato à operação de saneamento simples.

Soluções ERP Globais: SAP Business One e Microsoft Dynamics

No espectro generalista, o SAP Business One e o Microsoft Dynamics 365 representam o padrão da indústria para a gestão integrada de recursos (ERP), oferecendo módulos que cobrem desde a contabilidade à gestão de relacionamento com o cliente (CRM).

- **Aspetos Positivos:** Garantem a normalização de processos segundo as melhores práticas internacionais, escalabilidade quase infinita e uma integração nativa com ferramentas de produtividade (como o Office 365 no caso da Microsoft).
- **Limitações e Crítica:** Estudos sobre a implementação destas ferramentas em PMEs apontam frequentemente para um desajuste organizacional. Cichosz et al. (2020) alertam que a complexidade destas plataformas gera frequentemente resistência à mudança e exige longos períodos de formação. Além da barreira económica (elevados custos de licenciamento e manutenção), estas soluções sofrem de "rigidez de processos": obrigam a microempresa a adaptar o seu funcionamento ao software, em vez de o software servir a agilidade da empresa. Para a Palco d'Água, que necessita de registos rápidos no terreno, a interface destas soluções globais apresenta uma fricção operacional impeditiva.

Conclusão da Análise

A análise comparativa demonstra que, embora existam soluções robustas no mercado, estas falham em responder ao binómio Custo vs. Agilidade exigido por uma microempresa. A tabela abaixo sintetiza a lacuna de mercado que este projeto visa preencher.

Arquitetura de Sistema: Monólito Modular vs. Microserviços

Identificada a necessidade de uma solução à medida, a definição da sua arquitetura torna-se a decisão fundadora do projeto. A literatura atual contrapõe frequentemente duas abordagens:

- **Microserviços:** Esta arquitetura decompõe a aplicação em pequenos serviços independentes, comunicando via *APIs RESTful*. Embora ofereça escalabilidade horizontal, acarreta uma elevada complexidade de infraestrutura (necessidade de orquestração) e latência de rede, tornando-a inadequada para um ambiente de alojamento partilhado e para uma equipa de desenvolvimento reduzida.
- **Monólito Modular:** Para este projeto, optou-se pela arquitetura de Monólito Modular. Esta abordagem organiza o código em módulos lógicos (domínios de negócio) dentro de uma única base de código implantável.

Esta escolha justifica-se por três fatores determinantes:

1. **Integridade e Consistência (ACID):** Ao partilhar uma base de dados relacional unificada, o sistema garante nativamente a integridade referencial dos dados — impedindo serviços órfãos sem cliente associado — sem a complexidade de transações distribuídas.
2. **Desenvolvimento Ágil (RAD):** A utilização de um "monólito moderno" viabiliza uma metodologia *Rapid Application Development*. Ao eliminar a necessidade de desenvolver uma API intermédia separada do *frontend*, o tempo de desenvolvimento é drasticamente reduzido, permitindo cumprir os prazos académicos.
3. **Simplicidade Operacional:** Permite o *deploy* atómico na infraestrutura existente, facilitando a manutenção e reduzindo o *overhead* operacional.

Fundamentação Tecnológica

Definida a arquitetura, a modernização da Palco d'Água materializa-se numa *stack* tecnológica que privilegia a segurança, a rapidez e a eficiência de custos.

- **Laravel e o Ecossistema PHP:** O Laravel 11 destaca-se como uma das *frameworks* mais robustas para *backend*. O seu ORM (Eloquent) garante a abstração segura da base de dados MariaDB, enquanto as ferramentas nativas de autenticação (RBAC) asseguram o controlo de acessos por perfis.
- **React e a Reatividade do Frontend:** O React permite a construção de interfaces baseadas em componentes. A utilização de um *Virtual DOM* é essencial para a "Agenda Dinâmica", permitindo que o estado dos serviços se reflita instantaneamente na interface sem recarregar a página.
- **Inertia.js (A Ponte):** O Inertia.js atua como o elo de ligação desta arquitetura monolítica. Permite criar uma *Single Page Application* (SPA) mantendo a simplicidade de desenvolvimento do lado do servidor (Laravel), ideal para alojamento em cPanel onde APIs complexas aumentariam a latência.
- **Bootstrap 5 (Interface e Responsividade):** A adoção deste *framework* CSS visa acelerar a prototipagem e garantir uma experiência *mobile-first*. A sua grelha responsiva é crítica para assegurar que o CRM seja funcional em *smartphones*, permitindo aos técnicos consultarem e finalizarem serviços diretamente no local da intervenção.
- **Integração GeoAPI.pt:** Para suportar o sistema híbrido de moradas, a integração com a GeoAPI garantirá a normalização dos dados geográficos, operando com uma lógica de *cache* persistente na base de dados local para minimizar dependências externas.Desafios da Infraestrutura e Estratégias de Mitigação

A opção estratégica pelo alojamento na infraestrutura existente (cPanel), embora vantajosa economicamente, apresenta desafios técnicos que serão mitigados da seguinte forma:

- **Limitações de Processamento:** Para evitar timeouts do servidor, tarefas pesadas (como envio de emails) serão processadas de forma assíncrona através de Queues (Filas) do Laravel.

- **Deploy Seguro:** Para mitigar o risco de erro humano na atualização via FTP, será configurado um fluxo de integração contínua (CI/CD) para automatizar a transferência segura de ficheiros.
- **Segurança de Pastas:** A estrutura de pastas será adaptada para garantir que o código sensível e as credenciais (.env) permanecerão num nível acima da pasta pública, inacessíveis via navegador.

Análise Operacional: Processos Manuais vs. Automatização

Validada a tecnologia, importa quantificar o impacto esperado face ao cenário atual. O processo vigente caracteriza-se pela redundância de dados e pelo fenómeno de "dupla escrita" (papel + software), gerando uma carga administrativa excessiva. A Tabela seguinte apresenta a estimativa de impacto da nova solução:

Indicador	Processo Atual (Manual)	Solução Proposta (Digital)	Ganho de Eficiência
Registo em Obra	~5 min (Escrita manual da folha de serviço)	< 1 min (Seleção rápida e inputs automáticos)	80%
Processamento Admin.	~10 min (Decifrar caligrafia + Inserção manual no PC)	2 min (Validação visual e cópia direta)	80%
Tempo Total / Serviço	~15 minutos	~3 minutos	80%
Integridade de Dados	Baixa (Risco de erro na transcrição de NIF/Morada)	Elevada (Validação automática via GeoAPI e algoritmos)	Eliminação de erros

Tabela 1 - Análise Processos Manuais vs. Automatização

Metodologia

Para a operacionalização do projeto, adota-se uma metodologia de gestão híbrida, que concilia duas realidades distintas:

- **Estrutura Waterfall (Cascata):** Ao nível macro, o projeto segue o modelo clássico ou "em cascata" descrito por **Pressman e Maxim (2019)**, obedecendo ao rigor do calendário académico com fases sequenciais e marcos temporais inamovíveis. Esta estrutura garante o cumprimento dos prazos institucionais e a produção documental exigida.
- **Execução Ágil (Iterativa):** Dentro da fase de implementação técnica (Fase 3), abandona-se a rigidez sequencial em favor de uma abordagem Ágil. O desenvolvimento do software ocorrerá de forma iterativa e incremental, permitindo validar funcionalidades críticas (como a máquina de estados) através de ciclos curtos de feedback.

Esta estratégia híbrida mitiga o risco do "efeito túnel" — comum em projetos puramente em cascata, onde os erros só são detetados no final — assegurando que a solução entregue em junho é funcional, testada e perfeitamente ajustada aos requisitos operacionais do cliente.

Desenvolvimento

O desenvolvimento do sistema da Palco d'Água será realizado de forma estruturada, traduzindo os requisitos do SRS em funcionalidades concretas através da framework Laravel 11 e PHP 8.2+. A aplicação será construída com uma organização modular do código e componentes React, recorrendo ao Inertia.js para garantir uma interface reativa. Para a persistência de dados, utilizar-se-á o MariaDB, onde será definida a estrutura da base de dados com tabelas para clientes, serviços e viaturas, assegurando a consistência ACID. Durante o desenvolvimento, serão implementadas funcionalidades críticas como o sistema de autenticação por perfis, a máquina de estados para automação do ciclo de vida dos serviços e a gestão de frota com alertas de manutenção. A integração com a GeoAPI.pt para a normalização de moradas será cuidadosamente desenvolvida para garantir a eficácia e segurança da solução final.

Engenharia de Requisitos (SRS)

1. Público-Alvo e Sugestões de Leitura

Este documento destina-se às equipas de desenvolvimento, gestão e testes, bem como aos utilizadores que pretendam maximizar a sua experiência enquanto stakeholders. O documento poderá ser distribuído a outros intervenientes se a Palco d'Água assim o entender. O SRS descreve detalhadamente o projeto Palco d'Água, abrangendo o website institucional e o sistema ERP para gestão operacional e de frota, incluindo as características e os requisitos funcionais e não-funcionais acordados para a modernização tecnológica da empresa.

2. Âmbito do Projeto

O projeto Palco d'Água permitirá a transição de uma gestão analógica para um ecossistema digital integrado, resolvendo a ineficiência operacional e a dispersão de informação que atualmente afetam a empresa. Neste contexto, deverão ser cumpridos os seguintes propósitos:

- **Centralização de Informação:** Unificar os dados de clientes, serviços e faturação num sistema ERP seguro e acessível.
- **Automação de Serviços:** Implementar uma máquina de estados para monitorizar o ciclo de vida das intervenções em tempo real.
- **Monitorização de Frota:** Automatizar o registo de quilometragens e a emissão de alertas para manutenções preventivas.
- **Otimização de Dados:** Integrar a GeoAPI.pt para garantir a normalização de moradas e reduzir erros de introdução manual.

3. Glossário

ACID	Atomicity, Consistency, Isolation, Durability (Atomicidade, Consistência, Isolamento, Durabilidade)
API	Application Programming Interface (Interface de Programação de Aplicações)
CI/CD	Continuous Integration/Continuous Delivery (Integração Contínua/Entrega Contínua)
CRM	Customer Relationship Management (Gestão de Relacionamento com o Cliente)
CSS	Cascading Style Sheets (Folhas de Estilo em Cascata)
DER	Diagrama Entidade-Relação
DOM	Document Object Model (Modelo de Objeto do Documento)
ERP	Enterprise Resource Planning (Planeamento de Recursos Empresariais)
FTP	File Transfer Protocol (Protocolo de Transferência de Ficheiros)
GIS	Geographic Information System (Sistema de Informação Geográfica)
HTTP	Hypertext Transfer Protocol (Protocolo de Transferência de Hipertexto)
HTTPS	Hypertext Transfer Protocol Secure (Protocolo de Transferência de Hipertexto Seguro)
ISTEC	Instituto Superior de Tecnologias Avançadas
JSON	JavaScript Object Notation. Formato leve de troca de dados, fácil de ler por humanos e máquinas, utilizado para enviar informação entre o servidor e a aplicação móvel/web.
MVC	Model-View-Controller. Padrão de arquitetura de software utilizado pelo Laravel, que separa a lógica da aplicação (Controller) da interface (View) e dos dados (Model).
NIF	Número de Identificação Fiscal
ORM	Object-Relational Mapping (Mapeamento Objeto-Relacional)
PDF	Portable Document Format (Formato de Documento Portátil)
PHP	Hypertext Preprocessor (Pré-processador de Hipertexto)
PME	Pequenas e Médias Empresas
POO	Programação Orientada a Objetos
RAD	Rapid Application Development (Desenvolvimento Rápido de Aplicações)
RBAC	Role-Based Access Control (Controlo de Acesso Baseado em Funções)
RBAC	Role-Based Access Control. Método de regulação de acesso a recursos informáticos ou de rede com base nas funções de utilizadores individuais dentro da organização.
REST	Representational State Transfer (Transferência de Estado Representacional)

RGPD	Regulamento Geral de Proteção de Dados
SPA	Single Page Application (Aplicação de Página Única)
SQL	Structured Query Language (Linguagem de Consulta Estruturada)
TLS	Transport Layer Security. Protocolo criptográfico (sucessor do SSL) que garante a privacidade e integridade de dados entre duas aplicações em comunicação (usado no HTTPS).
UI	User Interface (Interface de Utilizador)
UML	Unified Modeling Language (Linguagem de Modelação Unificada)

Tabela 2 – Glossário

4. Referências

- **Cichosz, M., Wallenburg, C. M., & Knemeyer, A. M.** (2020). *Digital transformation at logistics service providers: Barriers, success factors and leading practices.* International Journal of Logistics Management, 31(1), 209–238. https://doi.org/10.1108/IJLM-08-2019-0229
- **cPanel, L.L.C.** (2026). *cPanel & WHM Documentation (Version 132)*. Disponível em: https://docs.cpanel.net/
- **GeoAPI.pt.** (2026). *API de Códigos Postais e Geolocalização de Portugal*. Disponível em: https://geoapi.pt/
- **GitHub, Inc.** (2026). GitHub Actions Documentation. Disponível em: https://docs.github.com/en/actions
- **Inertia.js.** (2026). *Build single-page apps, without building an API*. Disponível em: https://inertiajs.com/
- **Laravel LLC.** (2026). *Laravel 11 Documentation*. Disponível em: https://laravel.com/docs/11.x
- **MariaDB Foundation.** (2026). *MariaDB Server 11.4 Documentation*. Disponível em: https://mariadb.com/docs/
- **Meta Platforms, Inc.** (2026). *React - The Library for Web and Native User Interfaces*. Disponível em: https://react.dev/reference/react
- **Microsoft.** (2026). Microsoft Dynamics 365: Aplicações empresariais inteligentes. Disponível em: https://dynamics.microsoft.com/pt-pt/
- **Navia**. (2026). *Navia - Uma plataforma que une dados, sistemas, operações e pessoas num ecossistema digital único*. Disponível em: https://www.naviasolutions.com/
- **Pinto, H. R.** (2024). *Scrum Project Based Learning (PBL) na educação de licenciatura em Engenharia Informática.* PRATICA - Revista Multimédia de Investigação em Inovação Pedagógica e Práticas de E-Learning, 7(1), 91–99. https://doi.org/10.34630/pel.v7i1.5820
- **Pressman, R. S., & Maxim, B. R.** (2019). *Software Engineering: A Practitioner's Approach* (9th Edition). McGraw-Hill Education. ISBN: 978-1-259-87297-6.

- **Rupeika-Apoga, R., & Petrovska, K.** (2022). *Barriers to sustainable digital transformation in micro-, small-, and medium-sized enterprises.* Sustainability, 14(20), 13558. https://doi.org/10.3390/su142013558
- **SAP SE.** (2026). SAP Business One: ERP para pequenas empresas. Disponível em: https://www.sap.com/portugal/products/erp/business-one.html
- **The Bootstrap Authors.** (2026). *Bootstrap 5.3 Documentation*. Disponível em: https://getbootstrap.com/docs/5.3/
- **The PHP Group.** (2026). *PHP Manual (Version 8.2+)*. Disponível em: https://www.php.net/docs.php
- **Ulas, D.** (2019). *Digital transformation process and SMEs.* Procedia Computer Science, 158, 662–671. https://doi.org/10.1016/j.procs.2019.09.101
- **Vite.** (2026). *Next Generation Frontend Tooling*. Disponível em: https://vitejs.dev/

5. Organização deste Documento

Este documento de Especificação de Requisitos de Software (SRS) está estruturado de forma a apresentar uma progressão lógica, desde a visão geral do negócio até às especificidades técnicas da implementação. O documento organiza-se nas seguintes secções principais:

- **Secções 6 a 10 (Descrição Geral):** Definem o contexto macro do sistema, identificando os Atores intervenientes (Perfis de Acesso), as Funcionalidades chave do produto e as Restrições operacionais e de desenho que balizam o projeto.
- **Secções 11 a 15 (Interfaces Externas):** Detalham as interações do sistema com o mundo exterior, incluindo os requisitos de Hardware (sensores GPS), Software (APIs e Base de Dados) e protocolos de Comunicação (HTTPS/JSON).
- **Secção 16 (Requisitos Funcionais):** Constitui o núcleo do documento, especificando caso a caso as operações do sistema, desde a gestão de clientes até à faturação e auditoria, incluindo os fluxos de sucesso e exceção.
- **Secções 17 a 19 (Requisitos Não-Funcionais e Regras de Negócio):** Estabelecem os critérios de qualidade (Performance, Segurança, Usabilidade) e as regras lógicas que governam a integridade dos dados e o comportamento da aplicação.
- **Secções 20 a 22 (Planeamento e Futuro):** Encerram o documento com o roteiro técnico para a fase de implementação, definindo a arquitetura de classes, o esquema de base de dados e o cronograma de desenvolvimento.

Esta organização segue as recomendações da norma IEEE 830, adaptada à realidade de um projeto académico de Engenharia de Software focado em tecnologias Web modernas.

6. Classes de Utilizadores

O sistema de gestão de serviços de saneamento prevê a existência de três tipos de entidades (atores do sistema e stakeholders) e três perfis de acesso distintos, detalhados na tabela seguinte:

Ator/Stakeholder	Descrição	Perfil
Funcionário	Entidade operacional que consulta a agenda dinâmica, executa os serviços no terreno, opera a máquina de estados (registando o início e fim dos trabalhos) e reporta a quilometragem da frota.	Funcionário
Financeiro	Entidade responsável pela validação administrativa, com foco na transição dos serviços para os estados de "Faturado" e "Pago", e na extração de relatórios de faturação.	Financeiro
Administrador	Entidade com controlo total do sistema. Gere as contas de utilizadores, configura os parâmetros da Gestão de Frota (alertas de manutenção) e supervisiona todo o ciclo de vida dos serviços.	Admin

Tabela 3 - Descrição dos Atores e Perfis de Acesso do Sistema.

A tabela seguinte ilustra os tipos de entidades/perfis de acesso e respetivo nível de privilégios, que podem aceder ao sistema de informação **Sistema de Gestão de Serviços**, de modo a utilizarem as funcionalidades permitidas a cada nível (descrição detalhada nos casos de uso em secção adiante deste SRS).

Perfil	Descrição	Nível
Funcionário	Perfil de operação no terreno (Mobile). Tem acesso à agenda dinâmica, interação com a máquina de estados do serviço (iniciar/terminar) e registo de quilometragem da frota.	FU
Financeiro	Perfil administrativo (Backoffice). Tem acesso à validação de serviços concluídos ("Para Faturação"), controlo de pagamentos e extração de relatórios financeiros.	FI
Administrador	Perfil de gestão global (Backoffice). Possui acesso total ao sistema, incluindo gestão de utilizadores, configuração de alertas de frota e supervisão de todos os serviços.	AD

Tabela 4 - Perfis de Acesso e Níveis de Privilégio do Sistema.

De forma a respeitar possíveis situações de exceção relativamente às permissões dos perfis propostos na Tabela 2, sugere-se a modularização do sistema de informação em grandes unidades orientadas a objetivos, de maneira a poder gerir permissões não apenas ao nível do perfil, mas também ao nível do utilizador individual. Sugere-se assim a seguinte divisão modular:

Módulo	Descrição	Acesso
Execução de Serviços	Funcionalidades Mobile para o técnico consultar a sua agenda diária e operar a máquina de estados (check-in, registo fotográfico, check-out).	TE
Gestão de Agenda	Backoffice: Funcionalidades para criar novos serviços, atribuir equipas/viaturas e gerir o calendário global de intervenções.	AD, FI
Gestão de Clientes	Registo e gestão da ficha de clientes e contactos. Inclui a validação normalizada de moradas via integração com a GeoAPI.	AD, FI, TE
Gestão de Frota	Registo de viaturas, acompanhamento da evolução da quilometragem e gestão dos alertas automáticos de manutenção/inspeção.	AD
Histórico de Intervenções	Consulta e filtragem de todos os serviços realizados, organizados por cliente ou data, para análise operacional e relatórios.	AD, FI
Gestão Financeira	Validação de serviços terminados (estado "Para Faturação"), controlo de pagamentos e emissão de mapas para contabilidade.	FI, AD
Gestão de Utilizadores	Administração de contas de acesso, gestão de perfis (RBAC), auditoria de sistema e redefinição de credenciais.	AD

Tabela 5 - Divisão modular proposta para o Sistema de Informação

Esta classificação modular ao nível das permissões facilita a gestão de exceções no que diz respeito à gestão de acessos. Por exemplo, um perfil do tipo Financeiro (nível FI) não deverá ter (por omissão) acesso ao módulo de Gestão de Frota.

No entanto, esta divisão modular permite que um utilizador específico desse perfil possa ter permissões de acesso ao módulo referido (ex: para conferência de custos de manutenção), satisfazendo assim uma exceção que de outra forma não seria respeitada.

A figura seguinte ilustra a organização geral da aplicação Sistema de Gestão de Serviços e todos os componentes externos com os quais deverá interagir:

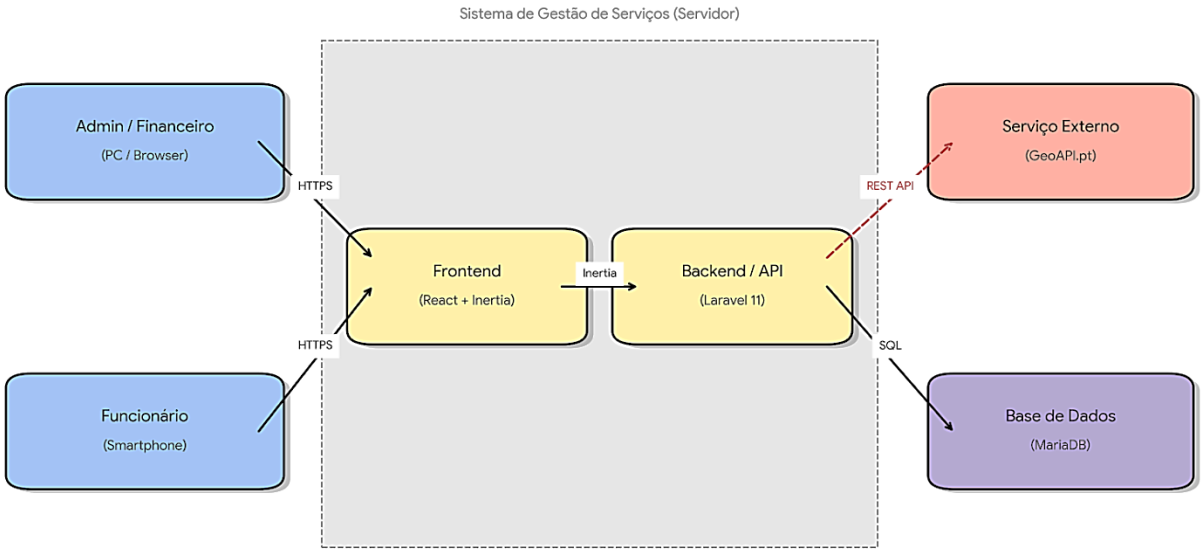

Figura 1 - Diagrama de Arquitetura do Sistema

7. Funcionalidades do Produto

O Sistema de Gestão de Serviços é uma aplicação web desenhada para apoiar a empresa na organização e automatização dos processos operacionais, focando-se no agendamento de serviços, gestão de clientes e manutenção da frota. O seu principal objetivo é substituir métodos manuais falíveis por uma solução centralizada, garantindo maior fiabilidade e eficiência através da redução de erros em moradas e estados de serviço.

Para tal, o sistema disponibiliza funcionalidades robustas, como a máquina de estados e validação geográfica via GeoAPI, cujo acesso é rigorosamente controlado por níveis de privilégio (RBAC). O sistema distingue requisitos globais de requisitos específicos por perfil, assegurando que cada utilizador acede apenas ao necessário. O capítulo 16 detalha, de forma exaustiva, todos os requisitos funcionais que compõem esta solução tecnológica.

8. Ambiente de Operação

A aplicação WEB Sistema de Gestão de Serviços foi projetada para operar primariamente no servidor atual da empresa (ambiente Linux com cPanel), mas com uma arquitetura flexível que permite a transição transparente para um modelo *Cloud* (SaaS/PaaS) sem necessidade de refatorização do código-base.

O desenvolvimento assenta numa *stack* tecnológica de última geração, combinando a robustez e segurança da *framework* Laravel 11 (PHP) no *backend* com a reatividade do React no *frontend*. A integração é feita via Inertia.js, proporcionando uma experiência de utilizador fluida e moderna, típica de uma *Single Page Application* (SPA), mantendo contudo, a organização lógica do padrão MVC. A persistência e integridade referencial dos dados são garantidas pelo Sistema de Gestão de Base de Dados MariaDB.

Para assegurar a interoperabilidade, o sistema implementa uma arquitetura capaz de consumir *APIs* externas (essencial para a integração com a GeoAPI.pt na normalização de moradas) e gerir pedidos internos de forma eficiente. Toda a comunicação de dados entre as camadas de *Frontend* (interfaces do utilizador) e o *Backend* (servidor) utiliza o padrão JSON. Esta abordagem garante não só a modularidade e escalabilidade do sistema, mas também uma elevada longevidade e facilidade de manutenção técnica a médio e longo prazo.

9. Restrições de Desenho e Implementação

O sistema será desenhado e implementado de modo a cumprir as recomendações de segurança, garantindo rigorosamente os requisitos do Regulamento Geral de Proteção de Dados (RGPD), dado o tratamento de dados pessoais (moradas, contactos e NIF) dos clientes.

A arquitetura impõe restrições técnicas específicas: a solução deve ser desenvolvida sobre a framework Laravel 11 (PHP) e React, exigindo um ambiente de hospedagem Linux (cPanel) com suporte a MariaDB para a persistência de dados. A comunicação entre o cliente (Mobile/Desktop) e o servidor, bem como a interoperabilidade com a API externa (GeoAPI.pt), será efetuada obrigatoriamente via protocolo HTTPS (encriptação TLS 1.2+), garantindo a confidencialidade e integridade da informação em trânsito.

10. Documentação para os Utilizadores

Estão previstos dois tipos de suporte documental para os diferentes perfis do Sistema de Gestão de Serviços:

1. Para a equipa técnica/administração: Será disponibilizado um manual técnico em formato digital, detalhando a estrutura da base de dados, os métodos do Backend Laravel e a especificação da integração com serviços externos (como a GeoAPI), permitindo a manutenção e escalabilidade futura do sistema.
2. Para os utilizadores finais (Técnicos e Financeiro): A aplicação privilegia a usabilidade intuitiva, substituindo manuais extensos por um sistema de ajuda contextual integrado na própria interface (tooltips e guias de navegação), que pode ser ativado ou desativado conforme o nível de experiência do utilizador na operação da agenda e da máquina de estados.

11. Pressupostos e Dependências

Pressupõe-se a disponibilidade de infraestrutura de hardware e software adequada ao desempenho do produto, especificamente um ambiente servidor **Linux** (como Ubuntu ou CentOS) configurado com servidor web (Apache ou Nginx) e suporte para **PHP 8.2+**, indispensáveis para a execução da framework **Laravel 11**.

Para a camada de dados, o sistema pressupõe a existência de uma instância **MariaDB**. Esta opção Open Source elimina custos de licenciamento, garantindo a escalabilidade e integridade referencial necessárias à transformação digital de microempresas sem onerar o orçamento operacional. Embora seja viável utilizar alojamento partilhado, recomenda-se que o servidor possua recursos de processamento suficientes para garantir a resposta rápida da API.

Funcionalmente, o sistema possui dependências críticas externas e do lado do cliente:

- **Conectividade:** É obrigatória uma ligação permanente à Internet para a comunicação com a **GeoAPI.pt** (validação de moradas) e sincronização em tempo real entre a App Mobile e o servidor.
- **Cliente:** A interface **React** requer navegadores modernos com JavaScript ativado.

- **Relatórios:** A visualização de faturas e fichas de serviço pressupõe a existência de um leitor de ficheiros PDF no dispositivo do utilizador.

12. Interfaces de Utilizador

Dado que o sistema está projetado para a otimização da gestão de serviços e frota, estão previstas as seguintes características fundamentais:

- **Controlo de acessos (RBAC):** Gestão rigorosa de permissões por perfil (Admin, Financeiro, Técnico), garantindo a segurança dos dados.
- **Interface e Localização:** Interface em Português (PT-PT), com design responsivo adaptado tanto a monitores de escritório como a ecrãs de *smartphones*.
- **Gestão Centralizada:** Unificação dos dados de clientes, histórico de intervenções, manutenção de viaturas e faturação numa única base de dados.
- **Usabilidade (UX):** Interface intuitiva focada na produtividade, especialmente na versão Mobile para os técnicos no terreno.
- **Integridade de Dados:** Validação automática de entradas (incluindo moradas via GeoAPI e lógica da máquina de estados), prevenindo erros humanos.

Sendo uma aplicação WEB baseada em **React** (SPA), qualquer navegador moderno que cumpra os requisitos do consórcio W3C é um cliente elegível, independentemente do sistema operativo (Windows, MacOS, Linux, Android ou iOS).

O sistema assegura uma operação híbrida fluida:

1. **Backoffice:** Otimizado para Desktop (PC/Portátil) para gestão administrativa.
2. **Frontoffice:** Otimizado para Mobile (Android/iOS) para registo de serviços e quilometragem no local.

Algumas operações, como a visualização de faturas ou fichas de obra geradas pelo sistema, podem depender da existência de software de leitura de ficheiros **PDF** no dispositivo do utilizador.

13. Interfaces de Hardware

Embora o sistema assente em frameworks web padronizados, a componente móvel (destinada aos Técnicos) pressupõe a interação com sensores de hardware específicos dos smartphones, nomeadamente o recetor GPS/GNSS. Esta interface é necessária para auxiliar na validação geográfica das moradas e na confirmação da presença do técnico no local da intervenção (quando aplicável). Para o perfil administrativo (PC), não estão previstas interfaces de hardware especiais além dos periféricos de entrada/saída padrão.

14. Interfaces de Software

Para o Sistema de Gestão de Serviços funcionar sem restrições e garantir a interoperabilidade, foram definidas as seguintes interfaces:

1. **API Interna (Backend):** A comunicação entre o Frontend (React) e o Backend (Laravel) assenta numa arquitetura RESTful, com troca de informação estruturada exclusivamente em formato JSON.
2. **Visualização de Documentos:** A geração de relatórios operacionais e faturas é feita em formato PDF. O dispositivo cliente deve possuir um navegador moderno com visualizador de PDF integrado ou software dedicado (ex: Adobe Reader) para a correta consulta destes documentos.
3. **Integração Externa (GeoAPI):** O sistema implementa uma interface de software para consumo da API externa GeoAPI.pt, permitindo a normalização e validação de códigos postais e moradas em tempo real.
4. **Migração de Dados:** Está prevista uma interface de importação (via ficheiros CSV/Excel) para permitir o carregamento inicial da base de dados (Clientes e Viaturas) a partir dos registos legados da empresa, garantindo a transição suave para o novo sistema.

15. Interfaces de Comunicação

Dada a natureza sensível dos dados processados (dados pessoais de clientes e faturação), a arquitetura de comunicação não é opcional:

- **Protocolo:** Toda a comunicação entre o Cliente (Browser/Mobile) e o Servidor, bem como entre o Servidor e a API externa, será realizada estritamente sobre o protocolo **HTTPS** (Porta 443), utilizando certificados SSL/TLS (versão 1.2 ou superior).
- **Segurança:** O sistema forçará o redirecionamento automático de pedidos HTTP inseguros para HTTPS. A comunicação assíncrona (AJAX/Axios) incluirá *tokens* CSRF (*Cross-Site Request Forgery*) nos cabeçalhos para garantir a autenticidade dos pedidos.

16. Requisitos Funcionais do Sistema

Esta secção descreve detalhadamente todos os requisitos funcionais do **Sistema de Gestão de Serviços** e a enumeração dos requisitos não-funcionais que possam estar relacionados com a funcionalidade.

Os requisitos funcionais descrevem as funcionalidades que o sistema deve disponibilizar aos seus utilizadores (Backoffice e Mobile). Estes requisitos especificam o que o sistema deve fazer, definem as ações que podem ser executadas (como agendamento ou mudança de estados), os dados que devem ser processados e as respostas que o sistema deve fornecer.

No contexto do **Sistema de Gestão de Serviços**, os requisitos funcionais definem as operações essenciais para a gestão operacional, técnica e financeira da empresa.

O diagrama de *Use Case* Geral está presente na secção de arquitetura de software.

O **Sistema de Gestão de Serviços** deverá permitir:

1. Visão de Gestão de Clientes

Campo	Descrição
Identificador	RF.01
Nome	Gestão de Clientes e Contactos
Atores	Admin, Financeiro, Técnico
Descrição	O sistema deve permitir criar, editar e consultar a ficha de clientes. Ao contrário de um sistema simples, este módulo suporta múltiplas moradas (Sede, Filial, Local de Obra) e múltiplos contactos por cliente, com validação automática de Código Postal.
Pré-Condições	O utilizador deve estar autenticado no sistema.
Pós-Condições	Cliente registado com pelo menos uma morada normalizada e pronto para associação a um Serviço.
Percurso Normal	1. O utilizador acede ao módulo "Clientes" e clica em "Novo". 2. O sistema solicita NIF e Nome. 3. Ao inserir o Código Postal o sistema deve verificar se a morada já existe na cache do sistema, caso não deverá integrar com a GeoAPI e preenche automaticamente Rua, Localidade e Distrito. 4. O utilizador insere o nº da porta e contactos (Email/Tel). 5. O utilizador grava a ficha do cliente.
Req. Não-funcionais	RNF.01 - A validação da morada via API externa deve ocorrer em menos de 2 segundos. RNF.02 - O sistema deve impedir a criação de clientes com NIF duplicado.

Tabela 6 - RF1. Gestão de Clientes

2. Visão de Gestão de Serviços e Agenda

Campo	Descrição
Identificador	RF.02
Nome	Agendamento e Criação de Serviços
Atores	Admin, Financeiro
Descrição	Permite registar um novo serviço no sistema, associando-o a um Cliente, a uma Tipologia de Serviço e alocando recursos (Viatura e/ou Equipa Técnica) numa data específica.
Pré-Condições	Cliente e Viatura devem estar ativos no sistema.
Pós-Condições	Serviço criado no estado inicial "Agendado".
Percurso Normal	1. O utilizador seleciona "Novo Serviço". 2. Pesquisa e seleciona o Cliente (o sistema carrega as moradas associadas). 3. Escolhe o "Tipo de Serviço" e a Morada de Intervenção. 4. Define a data e hora preferencial. 5. O sistema valida disponibilidade e permite alocar uma Viatura. 6. O utilizador confirma a criação do serviço.
Req. Não-funcionais	RNF.03 - O sistema deve impedir o agendamento de duas intervenções para a mesma viatura no mesmo horário (conflito de agenda).

Tabela 7 - RF2. Gestão de Serviços e Agenda

3. Visão Operacional (Máquina de Estados)

Campo	Descrição
Identificador	RF.03
Nome	Execução e Ciclo de Vida do Serviço (Mobile)
Atores	Técnico (via Mobile)
Descrição	Funcionalidade crítica que permite ao técnico no terreno controlar a evolução do serviço (Máquina de Estados). Inclui o registo de início, anexação de provas e fecho do serviço.
Pré-Condições	Serviço deve estar no estado "Agendado".
Pós-Condições	Serviço transita para o estado final "Pago" ou "Para Faturação".
Percurso Normal	1. O Técnico consulta a "Minha Agenda" no telemóvel. 2. Ao chegar ao local, clica em "Iniciar Serviço" (Estado muda para "A Decorrer"). 3. Realiza o trabalho e anexa fotos (upload para o servidor). 4. Ao finalizar, regista observações e seleciona o desfecho: "Cobrado no Local" (muda para "Pago") ou "Enviar para Faturação" (muda para "Para Faturação").
Req. Não-funcionais	RNF.04 - O upload de fotografias (provas de execução) deve ser otimizado para redes móveis (compressão automática).

Tabela 8 - RF3. Máquina de Estados

4. Visão de Gestão de Frota (Quilometragem)

Campo	Descrição
Identificador	RF.04
Nome	Registo de Quilometragem e Alertas
Atores	Técnico, Admin
Descrição	Registo da quilometragem das viaturas para controlo de custos e manutenção preventiva. Integra com o sistema de alertas.
Pré-Condições	Viatura ativa e autenticação do condutor.
Pós-Condições	Quilometragem atualizada na tabela Viatura; Alerta gerado se limite atingido.
Percurso Normal	1. O Técnico seleciona a viatura que está a conduzir. 2. Introduz o valor do odómetro atual. 3. O sistema valida se o novo valor é superior ao último registado. 4. O sistema verifica automaticamente se a quilometragem está abaixo 1.500km do limite de "Próxima Revisão". 5. Se o limite for atingido, ou a margem para a revisão for inferior a 1.500km o sistema cria um registo na tabela Alertas para o Admin.
Req. Não-funcionais	RNF.05 - O sistema deve manter um histórico imutável de todas as leituras de quilometragem por viatura para evitar adulteração.

Tabela 9 - RF4. Gestão de Frota (Quilometragem)

5. Visão de Gestão de Frota (Manutenções e Inspeções)

Campo	Descrição
Identificador	RF.05
Nome	Gestão de Manutenções e Inspeções
Atores	Admin
Descrição	Gestão do ciclo de vida da viatura, incluindo registo de revisões, inspeções (IPO) e reparações. Permite "dar baixa" dos alertas gerados.
Pré-Condições	Existência de alertas ou agendamento prévio.
Pós-Condições	Contador de "Próxima Revisão" reiniciado e alerta fechado.
Percurso Normal	1. O Admin recebe/consulta o alerta de "Revisão Necessária". 2. Regista a manutenção no sistema (data, custo, descrição). 3. O sistema atualiza a data/km da "Próxima Revisão" com base na configuração da viatura. 4. O alerta associado passa ao estado "Resolvido".
Req. Não-funcionais	RNF.06 - O sistema deve permitir anexar faturas digitalizadas (PDF/Imagem) ao registo da manutenção.

Tabela 10 - RF5. Gestão de Frota (Manutenções e Inspeções)

6. Visão Financeira e Faturação

Campo	Descrição
Identificador	RF.06
Nome	Validação e Controlo de Faturação
Atores	Financeiro, Admin
Descrição	Gestão dos serviços concluídos que aguardam processamento administrativo. Validação de dados fiscais antes da emissão da fatura.
Pré-Condições	Serviços no estado "Para Faturação".
Pós-Condições	Serviço transita para "Faturado" ou "Pago".
Percurso Normal	1. O Financeiro consulta a lista de serviços com estado "Para Faturação". 2. Verifica se a Ficha de Cliente tem NIF e Morada Fiscal válidos. 3. Emite a fatura no software externo e adiciona-a ao serviço (muda estado para "Faturado"). 4. Regista posteriormente o recebimento (muda estado para "Pago").
Req. Não-funcionais	RNF.07 - A lista de faturação pendente deve destacar visualmente serviços concluídos há mais de 30 dias.

Tabela 11 - RF6. Visão Financeira e Faturação

7. Visão de Segurança e Acessos

Campo	Descrição
Identificador	RF.07
Nome	Gestão de Utilizadores e Perfis (RBAC)
Atores	Admin
Descrição	Administração das contas de acesso, atribuindo perfis que limitam a visibilidade dos menus e funcionalidades.
Pré-Condições	Acesso de Administrador.
Pós-Condições	Novo utilizador criado ou perfil atualizado.
Percurso Normal	1. O Admin seleciona "Criar Utilizador". 2. Preenche nome, nome de utilizador e define uma password temporária. 3. Atribui o Perfil: "Técnico" (Acesso Mobile), "Financeiro" ou "Admin". 4. O sistema grava o utilizador e envia notificação de boas-vindas.
Req. Não-funcionais	RNF.08 - As passwords devem ser armazenadas na base de dados utilizando um algoritmo de hash seguro (Bcrypt).

Tabela 12 - RF7. Segurança e Acessos

8. Visão de Auditoria

Campo	Descrição
Identificador	RF.08
Nome	Auditoria de Ações e Sistema
Atores	Admin (Consulta), Sistema (Escrita)
Descrição	Registo automático e transversal de ações críticas para segurança e rastreabilidade (quem fez o quê e quando).
Pré-Condições	Qualquer alteração de dados sensíveis (Create/Update/Delete).
Pós-Condições	Registo criado na tabela Auditorias.
Percurso Normal	(Processo Automático de Backend) 1. O utilizador realiza uma ação crítica (ex: apagar cliente, mudar estado serviço). 2. O sistema interceta a ação e grava na tabela Auditorias: ID Utilizador, Data/Hora, Tipo de Ação e os dados alterados em formato JSON. 3. O Admin pode consultar estes registos no painel de controlo.
Req. Não-funcionais	RNF.09 - Os registos de auditoria não podem ser apagados ou alterados, nem mesmo pelo Administrador.

Tabela 13 - RF8. Auditoria de Ações e Sistema

17. Requisitos Não-Funcionais

Os requisitos não-funcionais descrevem as restrições e características de qualidade do sistema, definindo como este deve operar. Estes requisitos complementam os funcionais, assegurando que o sistema seja eficiente, seguro, fiável e adequado ao contexto de utilização.

1. Requisitos de Performance

A performance do sistema não deverá apresentar grandes desafios, pois à exceção dos documentos anexos (fotos de intervenção), os blocos de informação são pequenos (JSON).

- **Gestão de Imagens:** O carregamento de fotos de alta resolução (provas de execução) será sujeito a um redimensionamento e compressão automáticos no cliente (browser/app) antes do envio para o servidor, para poupar largura de banda e armazenamento.
- **Tempo de Resposta:** A API deve responder a pedidos de leitura em menos de 500ms e pedidos de escrita em menos de 1 segundo.
- **Sincronização:** A validação de moradas via GeoAPI deve ter um *timeout* definido de 5 segundos para não bloquear a interface.

2. Requisitos de Proteção

O **Sistema de Gestão de Serviços** garante a integridade dos dados e do ambiente onde opera:

- **Isolamento:** A aplicação não afeta dados fora do seu âmbito (base de dados própria) nem altera configurações do dispositivo do utilizador.
- **Interoperabilidade Segura:** A exportação de dados para contabilidade ou a comunicação com a GeoAPI é feita via canais seguros, sem expor a estrutura interna da base de dados.
- **Segurança Operacional:** O sistema deve bloquear interações complexas na interface Mobile se detetar movimento a alta velocidade (via GPS), desencorajando a utilização durante a condução.

3. Requisitos de Segurança

A segurança é prioritária dado o tratamento de dados pessoais (RGPD) e financeiros:

- **Autenticação:** O acesso é exclusivo a utilizadores registados. O sistema não permite registo público autónomo; todas as contas são criadas pelo Administrador.
- **Credenciais:** As palavras-passe devem ter complexidade mínima (ver RNF.19) e são armazenadas com hash robusto (Bcrypt).
- **Comunicação:** Todo o tráfego é obrigatoriamente cifrado via **HTTPS** (TLS 1.2+).
- **Proteção contra Ataques:** Implementação de *Rate Limiting* na API para prevenir ataques de força bruta no login e *Throttling* nas chamadas à GeoAPI para controlar custos.

4. Atributos de Qualidade do Software

- **Usabilidade (UX/UI):** A interface é desenhada com foco na produtividade. Utiliza-se a biblioteca **React** com componentes responsivos (via CSS moderno como Tailwind), garantindo compatibilidade total com *smartphones* (Android/iOS) usados pelos técnicos.
- **Offline First (Parcial):** Embora seja uma Web App, o sistema deve ser capaz de guardar temporariamente os dados de um formulário de serviço no *Local Storage* do navegador caso a rede falhe momentaneamente, submetendo-os assim que a ligação for restabelecida.

18. Requisitos Não-Funcionais das Regras de Negócio

ID	Regra de Negócio	Descrição
RNF.01	Unicidade do Cliente	O sistema deve impedir a criação de dois clientes com o mesmo NIF.
RNF.02	Validação de Morada	O sistema não pode permitir a criação de uma morada incompleta (sem Código Postal validado).
RNF.03	Alocação de Recursos	O sistema deve impedir o agendamento de uma viatura se esta já estiver ocupada nesse horário (conflito de agenda).
RNF.04	Consistência de Frota	O sistema deve impedir a inserção de uma quilometragem inferior à última registada para a mesma viatura.
RNF.05	Imutabilidade Fiscal	O sistema deve impedir a alteração ou eliminação de serviços que já estejam no estado "Faturado" ou "Pago".
RNF.06	Integridade do Serviço	Um serviço não pode ser fechado ("Pago") sem que tenha sido registada a data/hora de início e fim.
RNF.07	Dados Fiscais	O sistema impede a transição para "Para Faturação" se o cliente não tiver NIF válido.
RNF.08	Manutenção Preventiva	O sistema deve gerar um alerta automático para o Admin quando uma viatura estiver a 1.500km da próxima revisão.
RNF.09	Estado da Viatura	O sistema deve impedir a atribuição de serviços a viaturas marcadas como "Em Manutenção" ou "Inativa".
RNF.10	Provas de Execução	O sistema deve exigir pelo menos uma fotografia ou observação de texto antes de permitir fechar um serviço técnico.
RNF.11	Validação Temporal	A data de agendamento de um novo serviço não pode ser anterior à data/hora atual.
RNF.12	Sessão Segura	O sistema deve terminar a sessão automaticamente após 30 minutos de inatividade (timeout).
RNF.13	Política de Passwords	A senha deve ter no mínimo 8 caracteres, incluindo letras maiúsculas, minúsculas e números.
RNF.14	Auditoria	Nenhuma ação de eliminação de dados (Soft Delete) apaga o registo físico da base de dados; apenas marca como inativo.
RNF.15	Conectividade	Para operações críticas (GeoAPI, Sincronização), o dispositivo deve ter acesso ativo à Internet.

Tabela 14 - Requisitos Não-Funcionais

19. Outros Requisitos

Requisitos externos que influenciam a experiência de utilização:

- **Hardware:** Os dispositivos móveis dos técnicos devem possuir câmara funcional (para fotos), GPS (para validação de local) e ecrã tátil de pelo menos 5 polegadas.
- **Serviços de Terceiros:** O correto funcionamento da validação de moradas depende da disponibilidade do serviço externo GeoAPI.pt (SLA externo).
- **Email:** O envio de notificações automáticas (recuperação de senha, alertas de agenda) requer a configuração de um servidor SMTP válido.

20. Lista de Itens a Elaborar na Fase de Desenho

A abstração dos requisitos foi convertida em especificações técnicas concretas para a equipa de desenvolvimento, sendo elaborados os seguintes artefactos:

Arquitetura de Classes e Modelos (MVC):

- Definição dos **Models** (Eloquent ORM) que representam as entidades (Clientes, Serviços, Viaturas), incluindo os seus atributos, relações (1:N, N:N) e *casts* de dados.
- Definição dos **Controllers** e **Services**, separando a lógica de tratamento de pedidos da lógica de negócio.

Estrutura de Base de Dados (Schema):

- Diagrama Entidade-Relacionamento (ERD) final.
- Ficheiros de **Migrations** do Laravel para criação versionada das tabelas e índices.

<u>Serão ainda elaborados os seguintes artefactos:</u>

Interfaces de Integração (APIs):

- Especificação dos *Endpoints* da API REST interna (rotas, métodos GET/POST, códigos de resposta HTTP).
- Estrutura dos objetos **JSON** a transitar entre o Frontend (React) e o Backend, e na comunicação com a **GeoAPI.pt**.

Lógica de Negócio e Segurança:

- Lista de **Middlewares** para interceção de pedidos (verificação de *tokens*, gestão de sessões).
- Definição de **Policies** (Gateways) para controlo fino de permissões (ex: "quem pode apagar um serviço").

Protocolos de Segurança:

- Definição dos algoritmos de *hash* para palavras-passe (Bcrypt/Argon2).
- Configuração dos certificados SSL/TLS para garantir canais HTTPS.

21. Continuidade do Processo de Desenvolvimento

Após a conclusão da análise de requisitos do projeto Sistema de Gestão de Serviços, transposta neste documento de SRS, estabelece-se o seguinte itinerário para o ciclo de vida do desenvolvimento:

- **Validação do SRS:** Revisão e aprovação deste documento por parte dos stakeholders (Cliente/Orientadores) e equipa técnica.
- **Iteração de Ajustes:** Caso sejam detetadas inconsistências ou novas necessidades, será produzida uma revisão ao SRS (versão 1.1).
- **Prototipagem (UX/UI):** Criação dos *wireframes* e *mockups* das interfaces principais (Web e Mobile) para validação de usabilidade antes da programação.
- **Desenvolvimento (Sprints):** Início da implementação técnica seguindo a metodologia ágil, começando pela configuração do ambiente (Laravel/React) e estrutura da Base de Dados.
- **Testes e Deploy:** Fase de testes unitários e de integração, culminando com a instalação do sistema no servidor de produção.

22. Cronograma Proposto

Fase	Atividade / Tarefa	Meses (2025-2026)	Entrega / Milestone
I	Especificação de Interfaces e Segurança	fev	Documentação
II	Validação e Prototipagem	fev	Mockups
III	Implementação de Módulos Operacionais	mar - mai	Protótipo funcional
IV	Execução de Testes e Validação	mar - mai	Resultados de Testes
V	Escrita do Relatório, Poster e Vídeo	mai - jun	Entrega Final (06/jun)
Fim	Apresentação e Defesa Final	jun	Defesa (15-16/jun)

Tabela 15 - Cronograma Proposto

23. Plano de Projeto do Sistema de Gestão de Serviços

1. **Conceção e Modelação do Sistema**
   - Definir a arquitetura lógica (MVC com *Frontend* desacoplado via Inertia.js).
   - Elaborar os diagramas UML fundamentais (Casos de Uso, Classes e Entidade-Relacionamento).
   - Desenhar os *Wireframes* e *Mockups* das interfaces, diferenciando a vista *Desktop* (Admin) da vista *Mobile* (Técnicos).
   - Validar o fluxo de navegação e usabilidade (UX) antes da escrita de código.

2. **Implementação da Base de Dados**
   - Conceber o esquema relacional final adaptado ao MariaDB.
   - Criar os ficheiros de **Migrations** no Laravel para a criação estruturada e versionada das tabelas.
   - Implementar **Seeders** e **Factories** para povoar a base de dados com dados de teste (dummy data).
   - Definir as chaves estrangeiras e índices para garantir a integridade referencial e performance.

3. **Desenvolvimento da Aplicação**
   - **Backend (Laravel):** Configurar rotas, *Controllers* e *Models*. Implementar a lógica de autenticação e gestão de permissões (RBAC).
   - **Integrações:** Desenvolver o serviço de ligação à API externa (GeoAPI.pt) para normalização de moradas.
   - **Frontend (React):** Desenvolver os componentes visuais reutilizáveis e as páginas responsivas (Layouts PC e Mobile).
   - **Funcionalidades Mobile:** Implementar a lógica de captura de coordenadas GPS e *upload* de fotos no módulo técnico.

4. **Testes e Validação**
   - Realizar testes unitários e de integração (com *PHPUnit* ou *Pest*) às regras de negócio críticas.
   - Testar a responsividade da interface em diferentes dispositivos (Computador, Tablet, Smartphone).
   - Verificar a segurança (sanitização de *inputs*, proteção CSRF e acessos HTTPS).
   - Validar o ciclo completo de um serviço (do agendamento à faturação) para detetar falhas de fluxo.

5. **Documentação e Entrega Final**

- Elaborar a documentação técnica (Instalação no servidor Linux, Estrutura da API).
- Criar o manual de utilizador ou configurar o sistema de ajuda contextual na aplicação.
- Preparar o pacote de *deploy* e realizar a apresentação final do projeto.
- Validar o cumprimento dos objetivos iniciais face ao produto entregue.

Arquitetura de Software

Diagrama Use Case:

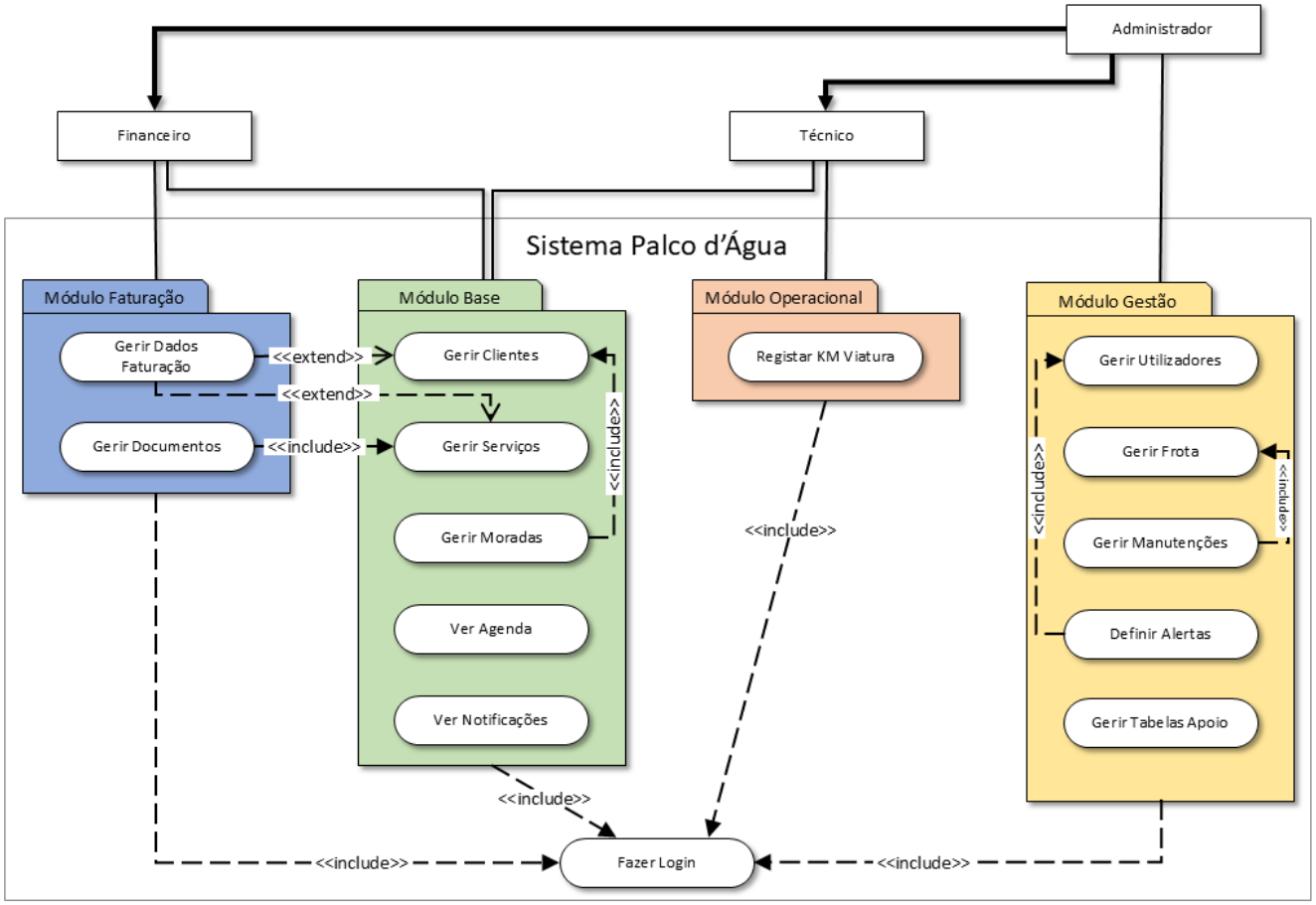

Figura 2 - Diagrama de Use Case

Diagrama de classes:

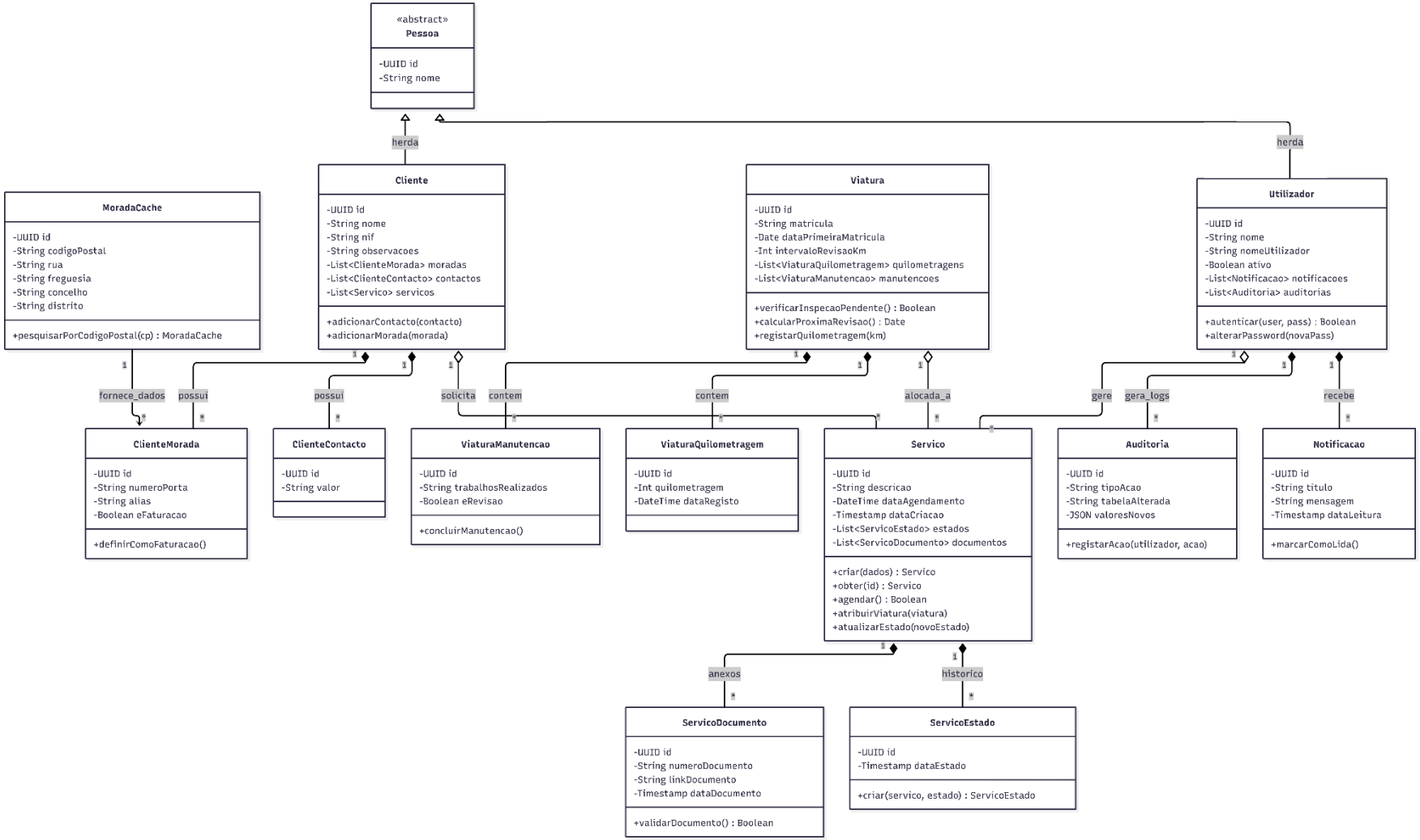

Figura 3 - Diagrama de Classes

Diagrama Entidade Relação (ER):

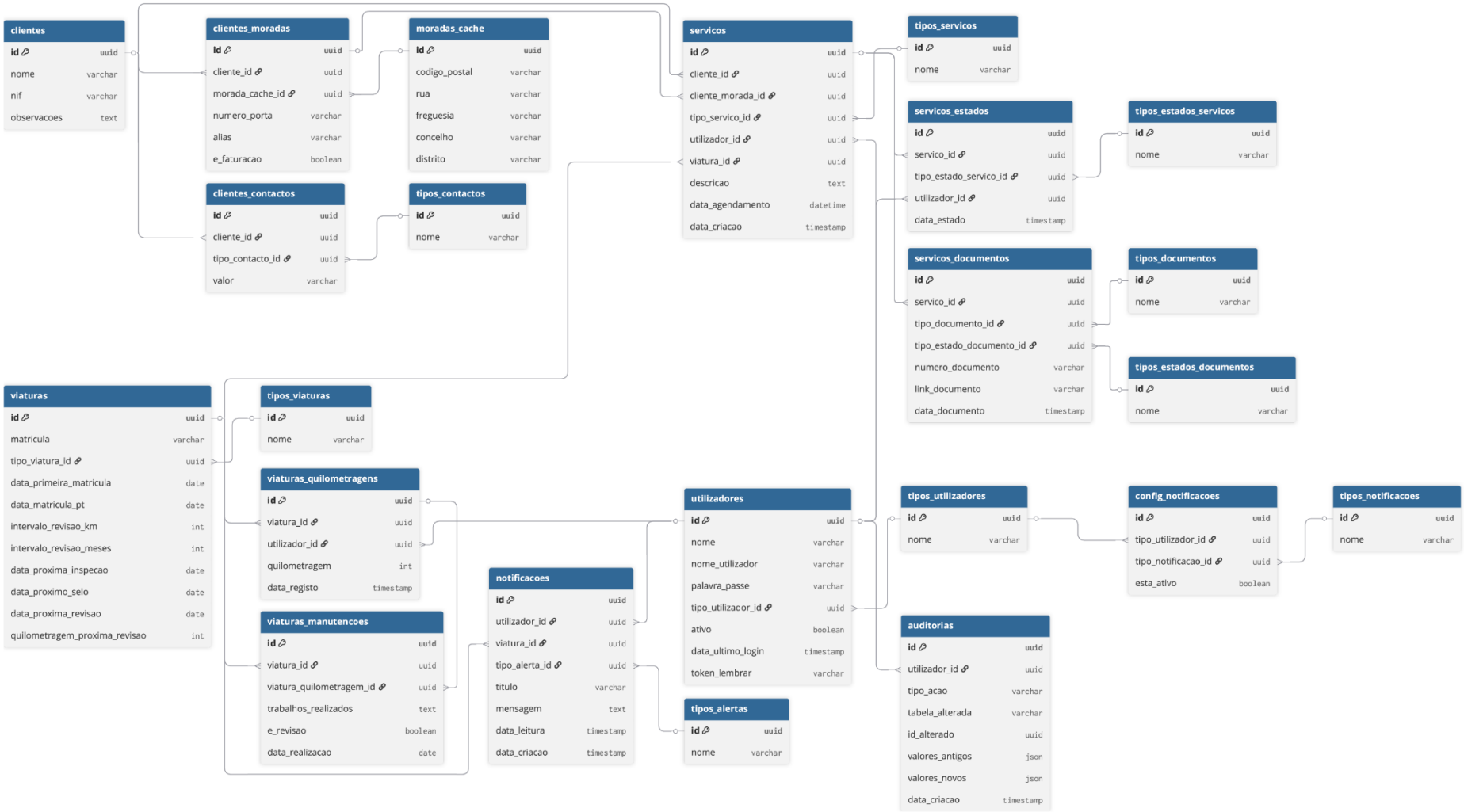

Figura 4 - Diagrama de Entidade Relação

Construção e testes de software

Para validar a capacidade de processamento do motor de base de dados MariaDB, foram realizados testes preliminares de inserção massiva de dados utilizando instruções SQL nativas e *Stored Procedures* (ver Figuras 5, 6 e 7).

Embora os resultados demonstrem uma elevada performance na execução direta de SQL, a arquitetura final do sistema optou pela utilização do *Eloquent ORM* (Laravel). Esta decisão estratégica privilegia a segurança (proteção automática contra *SQL Injection*), a facilidade de manutenção e a abstração do código, aceitando um *overhead* de processamento desprezível para o volume de dados esperado numa PME.

Classe GeradorBenchmark:

```php
<?php

namespace App\Classes;

class GeradorBenchmark
{
    private array $consoantes = ["b", "c", "d", "f", "g", "h", "j", "k", "l", "m", "n", "p", "q", "r", "s", "t", "v", "w", "x"];
    private array $vogais = ["a", "e", "i", "o", "u"];

    public function gerarNome(): string
    {
        $nome = "";
        for ($i = 0; $i < 2; $i++) {
            $palavra = strtoupper($this->consoantes[array_rand($this->consoantes)]) . $this->vogais[array_rand($this->vogais)];
            for ($j = 0; $j < rand(2, 4); $j++) {
                $palavra .= $this->consoantes[array_rand($this->consoantes)] . $this->vogais[array_rand($this->vogais)];
            }
            $nome .= ($i == 0 ? "" : " ") . $palavra;
        }
        return $nome;
    }

    public function gerarNifValido(): string
    {
        // Gera um NIF começado por 2 (comum em Portugal)
        $nif = [2];
        for ($i = 1; $i < 8; $i++) $nif[] = rand(0, 9);

        $checkDigit = 0;
        for ($i = 0; $i < 8; $i++) $checkDigit += $nif[$i] * (10 - $i - 1);

        $checkDigit = 11 - ($checkDigit % 11);
        $nif[] = ($checkDigit >= 10) ? 0 : $checkDigit;

        return implode('', $nif);
    }
}
```

Figura 5 - Construção Class para Testes

Controlador ClienteController:

```php
        // --- 1. TESTE SQL RAW ---
        $startSql = microtime(true);
        DB::beginTransaction();
        try {
            for ($i = 1; $i <= $total; $i++) {
                DB::insert("INSERT INTO clientes (id, nome, nif, observacoes, created_at, updated_at) VALUES (?, ?, ?, ?, NOW(), NOW())", [
                    (string) Str::uuid(), $gerador->gerarNome(), $gerador->gerarNifValido(), "Teste SQL"
                ]);
            }
            DB::commit();
        } catch (\Exception $e) {
            DB::rollBack();
            return response()->json(['success' => false, 'message' => 'Erro SQL: ' . $e->getMessage()], 500);
        }
        $timeSql = microtime(true) - $startSql;

        // --- 2. TESTE STORED PROCEDURE ---
        $startSp = microtime(true);
        DB::beginTransaction();
        try {
            for ($i = 1; $i <= $total; $i++) {
                DB::statement("CALL QClienteCriar(?, ?, ?, ?)", [
                    (string) Str::uuid(), $gerador->gerarNome(), $gerador->gerarNifValido(), "Teste SP"
                ]);
            }
            DB::commit();
        } catch (\Exception $e) {
            DB::rollBack();
            return response()->json(['success' => false, 'message' => 'Erro SP: ' . $e->getMessage()], 500);
        }
        $timeSp = microtime(true) - $startSp;

        return response()->json([
            'success' => true,
            'sql' => round($timeSql, 4),
            'sp'  => round($timeSp, 4),
            'qtd' => $total
        ]);
    }
```

Figura 6 - Construção Controller para Testes

View Resultados do Teste de Carga:

Figura 7 - Teste de inserção de dados na base de dados

Conclusão

O presente documento de Especificação de Requisitos de Software (SRS) estabelece as fundações teóricas e técnicas necessárias para o desenvolvimento do **Sistema de Gestão de Serviços**. Ao longo das várias secções, detalhou-se uma arquitetura robusta e escalável, baseada em tecnologias de mercado modernas (**Laravel, React e MariaDB**), capaz de responder aos desafios da transformação digital da empresa.

A solução proposta vai além da simples informatização de dados; ela reestrutura o fluxo operacional da organização. A integração de funcionalidades críticas como a validação geográfica de moradas (via **GeoAPI**), a monitorização da frota em tempo real e a disponibilização de uma interface móvel para os técnicos, garante a eliminação de redundâncias, a mitigação de erros manuais e uma gestão proativa da manutenção das viaturas.

Ficou igualmente demonstrado que o sistema cumpre os requisitos de segurança e privacidade indispensáveis (RGPD), assegurando a proteção dos dados sensíveis dos clientes e a integridade da faturação através de controlo de acessos (RBAC) e encriptação.

Com o âmbito funcional delimitado e a arquitetura técnica validada, reúnem-se as condições ideais para transitar da fase de análise para a fase de **Implementação e Desenvolvimento**, com a garantia de que o produto final será uma ferramenta de valor acrescentado, fiável e alinhada com os objetivos estratégicos da organização.